

Analysis of technical art

Overdrive

Aventix
Epitech Paris
February 20, 2026

Summary

OverDrive – Analysis of technical art

1- Formula 1: OpenF1 as a central brick

- 1- Data available via OpenF1
- 2- Access methods
- 3- Technical decision - F1

2- Rally (WRC): WRC API + community ecosystem

- 1- WRC API – Live timing
- 2- Community documentation (RallyDataJunkie)
- 3- Technical decision – Rally

3- WEC (Endurance): Live timing Al Kamel via WEC interface

- 1- Live timing WEC
- 2- Technical constraints
- 3- Technical decision – WEC

4- Comparative synthesis

5. Conclusion

This section presents the analysis of existing technical solutions for the collection and exploitation of motorsport data as part of OverDrive.

The objective is to identify, for each targeted discipline, the available data sources, their maturity, their mode of access, and to justify the technical choices chosen for the project's Go backend.

1- Formula 1: OpenF1 as a central brick

For Formula 1, the technical choice is clear: **exclusive use of OpenF1**, with subscription for live access. OpenF1 is now a modern solution, designed for developers, covering both live and history.

1- Data available via OpenF1

Live data (subscription required)

- Real-time positions
- Regularly updated gaps and offsets
- Lap and sector times
- Fastest lap, disallowed laps
- Pit information (duration, number of pit stops, tires)
- Race control messages (flags, Safety Car, VSC, incidents)
- Weather data (track/air temperature, humidity, wind, rain)
- Team radio and session events

Historical data (free)

- Sessions since 2023
- Detailed lap times
- Race events
- Telemetry
- Rankings

The granularity between live and historical is consistent, which facilitates integration into a common model.

2- Access modes

OpenF1 offers :

- REST endpoints (JSON/CSV)
- Real-time streams (REST, MQTT, WebSockets)

This allows direct integration into a Go backend via :

- net/http for REST calls
- Native WebSocket or MQTT client

No Python dependencies are required.

3- Technical decision - F1

- Backend Go directly consuming OpenF1
- No use of FastF1 (Python-oriented, redundant)
- No industrial provider like Sportradar (unnecessarily burdensome for the project)

OpenF1 covers 100% of the functional collection needs for OverDrive, while remaining agnostic of the language.

2-Rally (WRC): WRC API + community ecosystem

Unlike F1, the rally (WRC) does not have a direct equivalent of OpenF1.

However, there are:

- An officially undocumented WRC API
- A community ecosystem documenting its use

1- WRC API – Live timing

The WRC API exposes in JSON :

- Season Calendar
- Rally Structure (legs, sections, special stages)
- Entry List (cars, drivers, co-drivers, classes)
- Times per Special Stage
- Split Times
- Standings per Special Stage and Overall

These endpoints are not officially documented, but are used by the official web interface.

2- Community Documentation (RallyDataJunkie)

Projects like RallyDataJunkie :

- Identifies URLs
- Details parameters
- Shows how to reconstruct usable structures

They help avoid massive HTML scraping and favor cleaner JSON feeds.

3- Technical decision – Rally

- Backend Go directly consuming WRC JSON endpoints
- Structured parsing (Go structs + database persistence)
- HTML scraping only in the fallback if an endpoint changes

This approach limits fragility while remaining realistic for an academic project.

3- WEC (Endurance): Live timing AI Kamel via WEC interface

The WEC has a different configuration. The data is managed by AI Kamel Systems, and accessible via the WEC web interfaces. There is no bundled public API equivalent to OpenF1.

1- Live timing WEC

The site fiawec.com exposes :

- Live positions
- Number of laps
- Lap times
- Pit stops
- Results

AI Kamel also centralizes WEC results and other championships (ELMS, etc.).

2- Technical constraints

Absence of official public REST API :

- It's not possible to call a documented endpoint like in OpenF1.
- Data is transmitted via the web interface (internal JSON or specific feeds).

Two options :

1. Intercept the JSON streams used by the interface.
2. Scrape the HTML if necessary.

3- Technical decision – WEC

- Backend Go retrieving live data exposed via the Al Kamel / fiawec.com interface
- Priority given to internal JSON feeds if available
- HTML scraping as a fallback solution

The data is then standardized in the OverDrive model :

- Races
- Cars
- Laps
- Pit stops

No paid B2B solutions are being considered, because :

- Disproportionate cost
- Unnecessary complexity
- Live timing WEC sufficient for the project's use cases

4- Comparative summary

Discipline	Main source	Type of access	Coverage	Technical choice
F1/	OpenF1	REST, MQTT, WebSockets	Telemetry, timing, positions, stands, weather, radio, standings	Backend Go consuming OpenF1 directly
Rally (WRC)	WRC API + community docs	Undocumented JSON REST	Calendar, engaged, special time, splits, standings	Backend Go consuming identified JSON endpoints
WEC	Live timing fiawec.com / Al Kamel	Internal JSON / HTML	Live positions, times, stands, results	Backend Go structuring the recovered data

5- Conclusion

The analysis of the state of the art shows that:

Formule 1

OpenF1 is a unique, sufficient, and modern building block. It covers both live and historical data, without the need to use FastF1 or an industry-specific provider.

Rallye

The WRC API and community documentation (RallyDataJunkie) represent the accessible state of the art. The JSON-first approach with scraping as a fallback is consistent for a Go backend.

WEC

The state of the art relies on Al Kamel's live timing, as displayed on the official website. Backend-side data retrieval and normalization allow for data integration without the need for cumbersome B2B solutions.